

Labb X2: Programspråkstidsresa

Tasfia Alam & Carolina Frisk Garcia
tasfiaa@kth.se & cafg@kth.se

6 olika programmeringsspråk från olika årtionden har använts för denna laboration för att koda fram de 10 första Bernoullitalen med hjälp av given pseudokod.

50/60-tal: COBOL

Cobol har en tydlig hierarki som är lätt att följa, men är väldigt kanske lite onödigt lång då programmet delas upp i divisioner, sektioner, paragrafer, meningar, satser och karaktärer. På sätt och vis blir ju koden självdokumenterande då man alltid vet vart man befinner sig vilket ändå ger ett gott exempel på att Cobol tillhör den imperativa familjen. Att implementera Bernoulli- algoritmen gick ändå helt okej när man förstod grundstrukturen och syntaxen. Ex variabler definieras i *data divison*, där man sedan kan arbeta med de i *procedure divison*.

Det som var frustrerande var språkets känslighet för detaljer som inte hade något med logiken att göra. Ett felplacerat punkttecken eller inget mellanslag mellan en operator och en siffra kunde krascha kompilatorn, vilket tog en stund att hitta. Kompileringen var inte allt för svår tack vare uppgiftsbeskrivningen, kompilerar den till en binär fil (cobc -x -free) och sedan kör den.

Att skriva ett större program hade varit väldigt jobbigt. Detta då det inte finns riktiga funktioner med parametrar, därför måste man spara argumentet i en variabel, anropa "funktionen" separat och sedan läsa resultaten från den variabeln.

70-tal: Smalltalk

Det som var positivt med Smalltalk var att pseudokoden kunde översättas ganska ordagrant. Motsvarande till funktioner finns block, där kod inom hakparenteser kan bli tilldelade variabler och köras vid behov. Med bra sökningar och läsning av uppgiftsbeskrivningen kunde man hitta relevant information, såsom att man kan bygga upp en array dynamiskt med `OrderedCollection` samt att `fraction` gav exakta svar istället för avrundade flyttal.

Det som var svårare att komma igång med var att allt i Smalltalk är ett objekt. Allt från enkla beräkningar till block är objekt som kommunicerar via meddelanden. Detta var nog det svåraste, nämligen att ändra sitt tankesätt och tänka på objekt och meddelanden. Att skriva ett större program hade nog tagit en stund att komma igång med då man behöver tänka på ett helt annat sätt än vad vi är vana vid att göra.

80-tal: Erlang

Att direkt översätta den imperativa pseudokoden till Erlang var väldigt svårt eftersom det är funktionellt dvs tillhör den deklarativa familjen så man kunde inte använda for-loopar utan behövde skapa hjälpfunktioner som är rekursiva. Därför har vi delat upp koden i både basfall och det rekursiva steget.

Listor i Erlang är vanliga men de är 1-indexerade men nästan alla språk vi känner till sen tidigare är 0-indexerat. Däremot, koncept som vi har lärt oss från Haskell och det funktionella paradigmet, gjorde kodskrivandet inte helt hopplöst då vi kunde använda vissa uppslag såsom högre ordningens funktion *foldl*. Det påminner också mycket om prolog, till exempel mönstermatchning, att det endast tillåter små bokstäver för variabler och att man har komma-tecken efter varje rad och punkt på den sista, vilket underlättade det syntaxmässigt lite grann.

Det var inte alltför svårt att kompilera på programmet då man redan får veta om kommandot *erl* i uppgiftsbeskrivningen som tar oss in till erlangs körningssystem, men senare var det klurigt att veta hur man skulle köra och printa de 10 första bernoulli talen. Till exempel behöver man definiera högst upp vilka funktioner från koden vi vill anropa.

90-tal: PHP

Eftersom vi har gjort klart W1 labben är vi vana med PHP syntaxen och det var väldigt lätt att översätta direkt från pseudokoden. Det enda man behövde lägga till var '\$'-tecknet framför varje variabel, vilket kunde vara lite krävande. Dessutom är både pseudokoden och PHP imperativa språk. Likt pseudokoden delade vi upp det i två funktioner: huvudfunktionen *B* räknar ut det *n*:te Bernoulli talet med hjälp av en nästlad for-loop, och funktionen *BINOM* som ger oss olika sätt man kan välja *k* element ur *n*, alltså en kombination.

Ett exempel på ett större projekt man kan göra med språket är webbprogrammering och server-side-programming (som vi har sett i W1 labben), till exempel: kommunikation med databaser, säkerhetskrav och lösenordshantering för webbsidor såsom inloggning, webbsidor där innehåll ändras beroende på vem som loggar in och mycket mer.

00-tal: Clojure

Clojure var mycket mer ansträngande än PHP, vilket inte var förväntat eftersom den ska vara mer modern. Anledningen till detta var mest pga syntaxen då den bygger på Lisp som har massor av parenteser och det var jobbigt att hålla koll på vilka klauuser som tillhör vad. Som exempel har vi 7 höger parenteser på en och samma rad. En snabb google sökning avslöjar att clojure tillhör det funktionella paradigmet och är dynamisk. Eftersom Erlang också är ett funktionellt språk och kodades innan clojure kunde vi ta inspiration från den, specifikt för rad

6 i pseudokoden, där vi har byggt upp den med en ackumulator. Oföränderlighet och rekursion är också vanligt i språket och underlättade en hel del, då vi tidigare har jobbat mycket med Haskell där dessa koncept är ganska vanliga.

10-tal: Rust

Rust var ett språk där det kändes naturligt att översätta pseudokoden till riktiga funktioner med parametrar som kunde anropas direkt i ett uttryck. Dessutom var det också smidigt att initiera en array med rätt storlek genom vektorer.

Det som upplevdes som svårast var typsystemet i Rust. Genomgående behövde man tänka om en variabel var `u16`, `usize` eller `f64` och explicit tänka till om man skulle märka det eller inte. Tack vare bra felmeddelanden av kompilatorn gick det bra att felsöka. Att skriva ett större arbete i Rust skulle nog inte vara en alltför stor omställning. Detta eftersom det var så liknande logik som Java som vi är vana vid ex, att man definierar funktioner och kallar dem i en `main` funktion.

Reflektion

Överlag var det en rolig uppgift att upptäcka språk som vi för det mesta aldrig har arbetat med innan. Vi gick på känsla och följde ingen särskild ordning när vi valde vilket språk vi skulle ge ett försök närmast. I överlag kan det konstateras att programmeringsspråken med fler år på nacken tog längre tid att förstå då logiken och/eller syntaxen var så annorlunda från de språk vi är vana med.